



EG1113 - C Programming for Engineering Lab

Report #4

Keyboard Input

Date of the experiment: September 11, 2024

Due date of report: September 13, 2024

Name: Nicholas Galus

1. Core concepts learned

Summarize the core concepts learned and practiced in this laboratory.

- Setup remote desktop to utilize the assigned Raspberry Pi 400
- Established network file sharing between the Raspberry Pi 400 and laptop utilizing Caja
- Created a hello world program in the C programming language

2. List of equipment and software used

- Raspberry Pi 400
- Notepad
- Lenovo Laptop
- Caja File Network Share

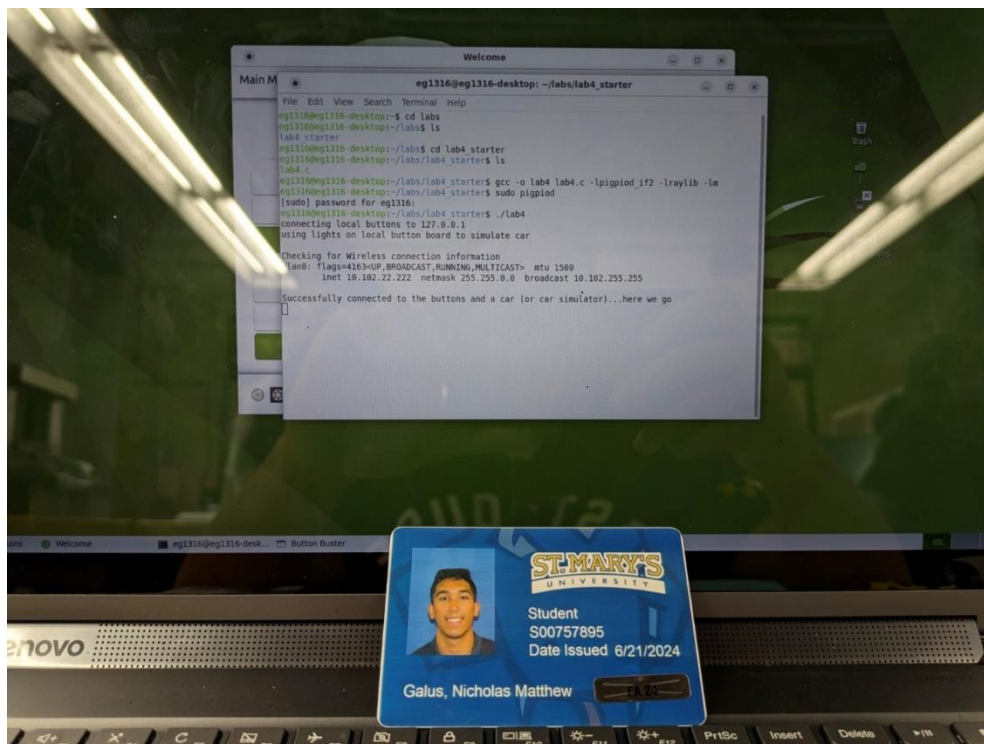
3. Relevant web links and other documents:

- Utilized starter code

4. Exercise Run Program from Terminal

4.1 Summary

- Run the starter program in the terminal to ensure that the program works
- Lab verification:



4.2 Procedures

1. Run and compile starter code

```

eg1316@eg1316-desktop: ~/labs/lab4_starter
File Edit View Search Terminal Help
eg1316@eg1316-desktop:~$ cd labs
eg1316@eg1316-desktop:~/labs$ ls
lab4_starter
eg1316@eg1316-desktop:~/labs$ cd lab4_starter
eg1316@eg1316-desktop:~/labs/lab4_starter$ ls
lab4.c
eg1316@eg1316-desktop:~/labs/lab4_starter$ gcc -o lab4 lab4.c -lpigpiod_if2 -lraylib -lm
eg1316@eg1316-desktop:~/labs/lab4_starter$ sudo pigpiod
[sudo] password for eg1316:
eg1316@eg1316-desktop:~/labs/lab4_starter$ ./lab4
connecting local buttons to 127.0.0.1
using lights on local button board to simulate car

Checking for Wireless connection information
wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
        inet 10.102.22.222 netmask 255.255.0.0 broadcast 10.102.255.255

Successfully connected to the buttons and a car (or car simulator)...here we go

```

5. Exercise Run Program in VS Code

5.1 Summary

- Making a program that utilizes the protoboard and setting up the Json file to run successfully in VS code

5.2 Procedures

1. Run the starter program without any modifications in VS Code

```

C:\lab4.c > ...
1 // LAB03 tester code
2 #include <stdio.h>
3 #include <pigpio_if2.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include "raylib.h"
7
8 int L_FWD=19, L_BK=26, R_FWD =16, R_BK=20, L_FWD_IN=27, L_BK_IN=5, R_FWD_IN=15, R_BK_IN=7; // pin assignments - for buttons
9
10 int main(int argc,
11 {
12     int pi_car, pi_light;
13     int saved_value;
14     int buttons, save;
15     int recording = 0;
16
17     // connect to the pigpiod (the server for gpio) that is on this pi --note that in the internet world, ip address 127.0.0.1 is lo
18     printf("connecting local buttons to %s\n", "127.0.0.1" );
19     pi_400 = pigpio_start("127.0.0.1", "8888");
20
21     if(argc > 1)
22     {
23         printf("connecting to lights (or car at) %s\n", argv[1] );
24         pi_car = pigpio_start(argv[1], "8888");
25     }
26
27     while(1)
28     {
29         // read buttons
30         buttons = pigpio_read(pi_400);
31         // read car
32         car = pigpio_read(pi_car);
33         // print status
34         printf("Buttons: %d, Car: %d\n", buttons, car);
35     }
36 }

```

Visual Studio Code

The preLaunchTask 'C/C++: gcc-10 build active file' terminated with exit code -1.

☐ Remember my choice in user settings

Show Errors Abort Debug Anyway

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

C/C++: gcc-10 build active file

```

rwx P a df: Ysno a 0 0 dwPc dwP ciwxiax dwP .n033: rafah aaf xahaxoana is 'Begin rawing'
rwx P a df: Ysno a 0 0 dwPc dwP ciwxiax dwP .n000: rafah aaf xahaxoana is 'gpio write'
rwx P a df: Ysno a 0 0 dwPc dwP ciwxiax dwP .n000: rafah aaf xahaxoana is 'gpio write'
rwx P a df: Ysno a 0 0 dwPc dwP ciwxiax dwP .n00 : rafah aaf xahaxoana is 'gpio write'
collect2: error: ld returned 1 exit ciwtus
Build finished with error(s).

```

2. Modify tasks.json file to be able to run in VS Code

```

1 {
2     "tasks": [
3         {
4             "type": "cppbuild",
5             "label": "C/C++: gcc-10 build active file",
6             "command": "/usr/bin/gcc",
7             "args": [
8                 "-fdiagnostics-color=always",
9                 "-g",
10                "-Wall",
11                "-O5",
12                "-pthread",
13                "${fileDirname}/*.c",
14                "-o",
15                "${fileDirname}/${fileBasenameNoExtension}",
16                "-lrt",
17                "-lGL",
18                "-lpigpiod_if2",
19                "-lraylib",
20                "-lm",
21                "-lpthread",
22                "-ldl",
23                "-lX11"
24            ],

```

6. Exercise Understanding Program

6.1 Summary

- Google RTFS
- Read the comments within the starter program to understand the program and create a table with the input and output
- Lab verification:



6.2 Procedures

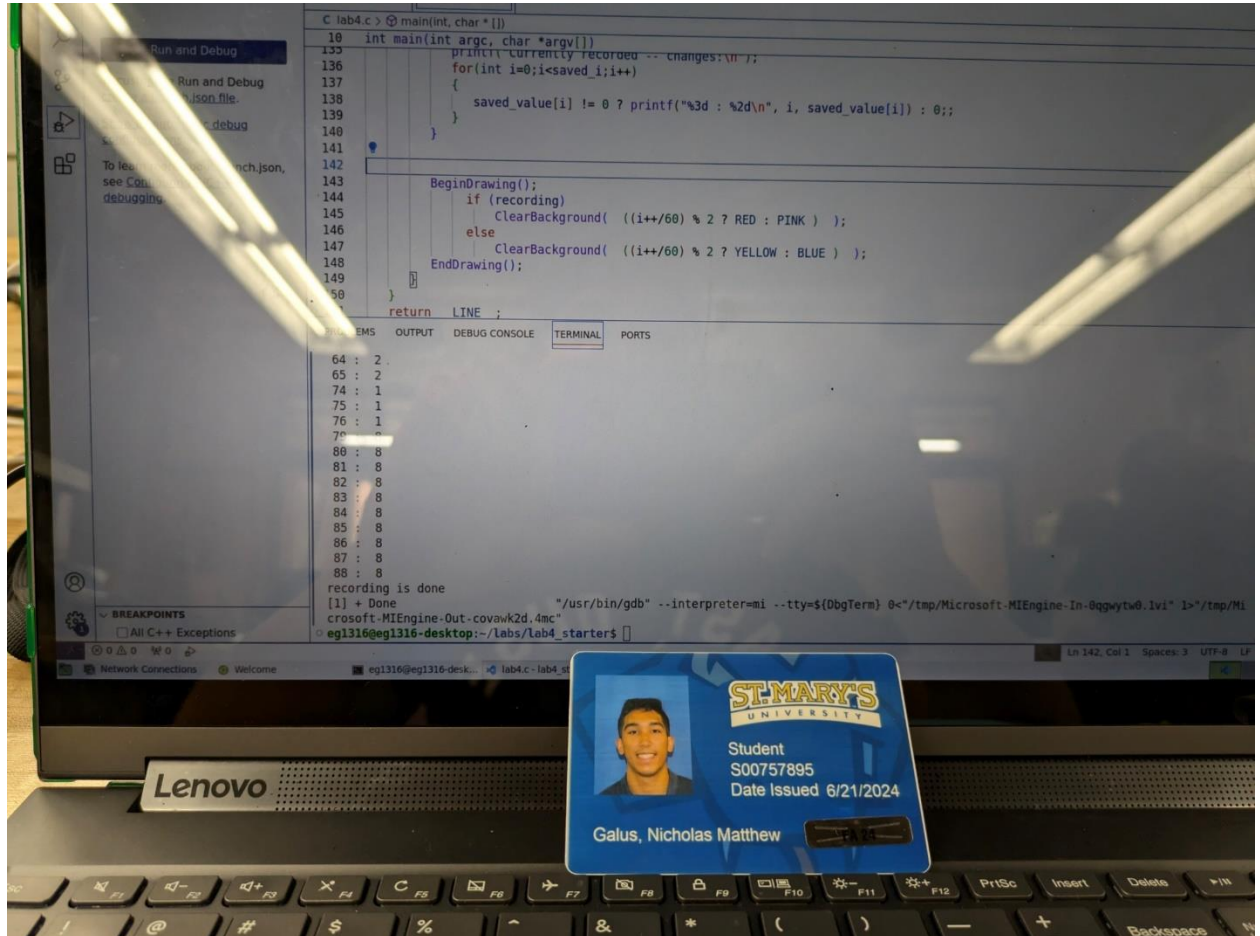
1. what RTFS means in software circles
RTFS stands for Read The Freakin' Source
2. Create table of input and output

Input provided by user	Action and output of program
W	Left forward light turns on
Q	Right forward light turns on
S	Left backward light turns on
A	Right backward light turns on
Q+W	Both forward light turns on
A+S	Both reverse lights turns on
No/Invalid input	Vehicle does not move (all lights are off).
R key is pressed	Begins recording and changes flashing colors to pink/red. The recorded inputs are saved for up to 5 seconds.
L key is pressed	Lists the recorded sequence of button presses, including any values that did not change, in the console output.
Output that happens at program startup	Creates a window with yellow and blue flashing background, indicating it is not recording (Idle state).
Output that happens periodically (e.g. the BLUE, YELLOW, RED, PINK colors that appear based on time and state)	The program records button inputs for up to 5 seconds and flashes the background between red and pink during this time. - If not recording: Background alternates between BLUE and YELLOW every second. - If recording: Background alternates between RED and PINK every second.

7. Exercise Modify Program to display recorded changes

7.1 Summary

- Modify the starter program for when the c key is pressed it outputs the changed values
- Lab verification:



7.2 Procedures

1. Create a new check statement to see when the user presses C

```
if (IsKeyDown(KEY_C)) // list the true values that were recorded
{
    printf("Currently recorded -- changes:\n");
    for(int i=0;i<saved_i;i++)
    {
        saved_value[i] != 0 ? printf("%3d : %2d\n", i, saved_value[i]) : 0;;
    }
}
```

2. Run the program and view the changed values

```
Successfully connected to the buttons and a car (or car simulator)...here we go
recording started
recording is done
Currently recorded -- changes:
60 : 8
61 : 8
62 : 8
63 : 8
64 : 8
65 : 8
66 : 8
67 : 8
68 : 8
69 : 8
110 : 1
111 : 1
112 : 1
113 : 1
```

8. Exercise Possible Modifications

8.1 Summary

- Think of possible modifications to improve the controller program

8.2 Ideas

- Turning on a dime by utilizing q+s (right) and w+a (left)
- Having a feature to have it stop more abruptly, by applying opposite force automatically when stopping
- Ability to coast to a stop
- Clutch and flywheel, to dump the clutch on start for quicker acceleration
- Using an analog input instead of a digital input to have more precision, set the throttle to a certain % instead of always full speed, or have more control when turning

9. Problems experienced and areas of concern

- No noteworthy problems occurred

